

Git, VS Code, and GitHub Workflow Setup

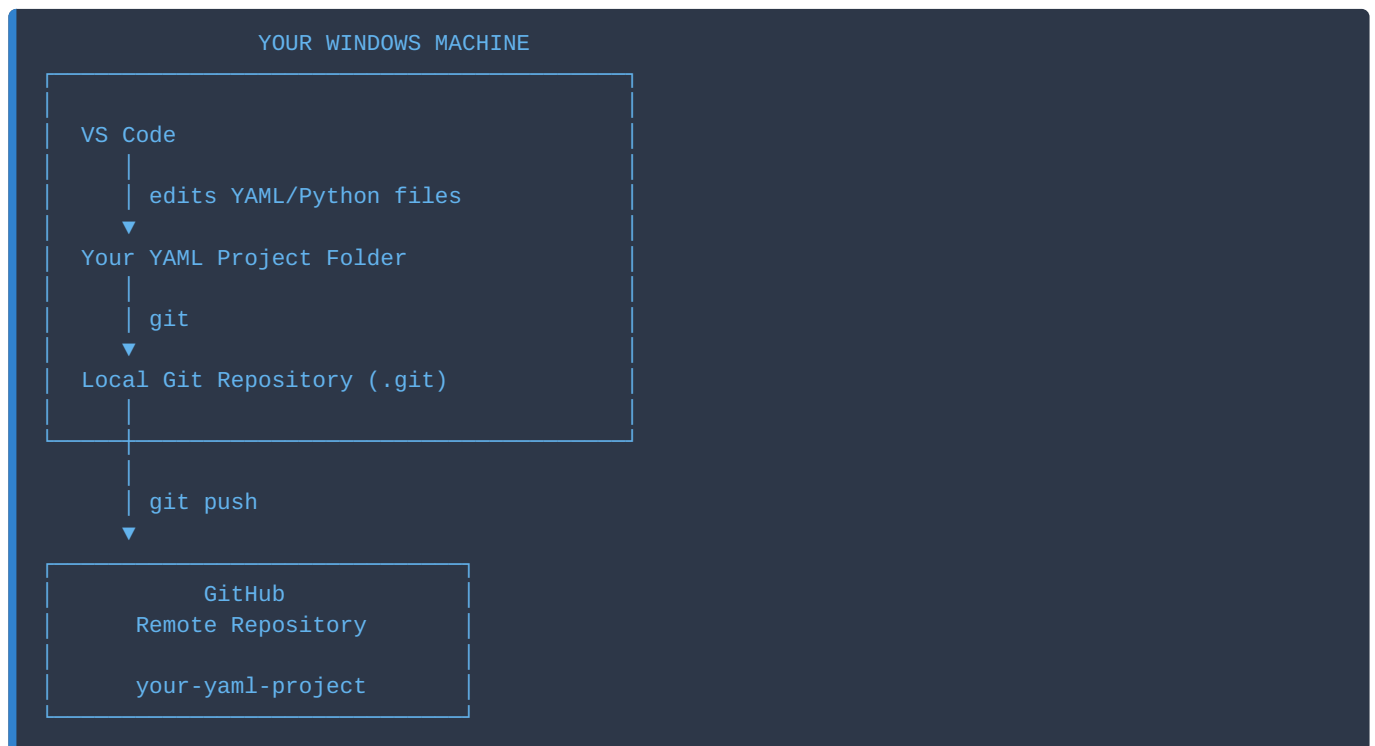
A Comprehensive Guide for Local & Remote Version Control Integration

Absolutely. Since you already have Git installed on Windows and VS Code installed, you are very close.

The important thing to understand first is:

VS Code does not need a separate “Git integration” installation. VS Code has built-in Git/Source Control support and uses the Git installation on your Windows machine. (Visual Studio Code)

For your YAML learning projects, I recommend using GitHub as the remote repository. The complete flow will look like this:



We'll set this up properly from the beginning, rather than just making it work temporarily.

1. First understand Git vs GitHub vs VS Code

These three things are different.

Git

Git is the version control software installed on your Windows machine.

It tracks:

```
file created
↓
file modified
↓
file committed
↓
file history maintained
```

For example:

```
settings.yaml
```

```
Version 1
```

```
↓
```

```
Version 2
```

```
↓
```

```
Version 3
```

```
↓
```

```
Version 4
```

VS Code

VS Code is your development environment.

You will write:

```
application:
  name: yaml-demo
  version: 1.0
```

inside VS Code.

VS Code then provides a graphical interface for Git operations such as staging, committing, branching and syncing. (Visual Studio Code)

GitHub

GitHub is the remote/cloud repository where you can store your Git repository.

Think:

```
Git
↓
Local repository

GitHub
↓
Remote repository
```

Git and GitHub are therefore not the same thing.

2. Verify Git installation

Open PowerShell or the VS Code terminal.

In VS Code:

```
Terminal
↓
New Terminal
```

Run:

```
git --version
```

You should get something similar to:

```
git version 2.x.x
```

If you see that, Git is available to Windows and VS Code can normally use it.

VS Code's official documentation requires Git to be installed on the machine; VS Code then detects and uses that installation. (Visual Studio Code)

3. Configure your Git identity

This is a one-time setup.

Git needs to know:

```
Who is making this commit?
```

Run:

```
git config --global user.name "Your Name"
```

For example:

```
git config --global user.name "VeeraSekhar"
```

Then:

```
git config --global user.email "your-email@example.com"
```

Use the email associated with your GitHub account if you want your GitHub commits associated with that account. VS Code's Git documentation also recommends configuring `user.name` and `user.email` before committing. (Visual Studio Code)

Check:

```
git config --global user.name
```

and:

```
git config --global user.email
```

You can also see everything:

```
git config --global --list
```

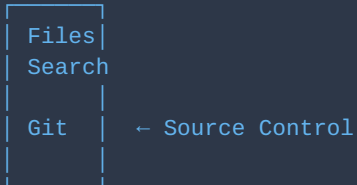
4. Check whether VS Code sees Git

Open VS Code.

Look at the left-side Activity Bar.

You should see the Source Control icon.

Usually it looks approximately like:



Click it.

Alternatively:

```
ctrl + Shift + G
```

This opens the Source Control view. (Visual Studio Code)

If Git is correctly detected, VS Code will show Git-related functionality.

5. Now let's connect your existing YAML project

This is probably the most important part for your situation.

You already have a YAML learning project.

For example, based on your current YAML learning project, you have something like:

```
yaml-project/
|
|— config/
|   |— development.yaml
|   |— production.yaml
|   |— application.yaml
|
|— src/
|   |— yaml_loader.py
|   |— config_loader.py
|   |— settings.py
|   |— __init__.py
|
|— main.py
|
|— ...
```

You should make this project folder a Git repository.

6. Open your YAML project in VS Code

In VS Code:

```
File
↓
Open Folder
↓
Select your YAML project folder
```

For example:

```
C:\Users\YourName\yaml-project
```

After opening it, your VS Code Explorer should show:

```
yaml-project
|
├── config
├── src
├── main.py
└── ...
```

7. Initialize Git in your YAML project

There are two ways.

Method A — VS Code GUI

Open:

```
Source Control
```

You should see something similar to:

```
Initialize Repository
```

Click it.

VS Code will execute the equivalent of:

```
git init
```

VS Code officially supports initializing an existing local project as a Git repository. (Visual Studio Code)

Method B — Terminal

I actually recommend learning this method because you are going to work with Git professionally.

Open VS Code terminal.

Make sure you are inside your project directory.

Run:

```
git init
```

You should see something similar to:

```
Initialized empty Git repository in ...
```

Now your project contains a hidden directory:

```
yaml-project/
|
├── .git/
├── config/
├── src/
├── main.py
└── ...
```

What is .git?

This is extremely important.

`.git` contains Git's internal repository information and history.

Think:

```
Your files
+
.git
↓
Git repository
```

Do not manually modify or delete `.git`.

8. Create a `.gitignore`

This is particularly important for your Python/YAML projects.

You don't want Git to upload things like:

```
__pycache__
.venv
.env
```

For example:

```
yml-project/
|
├── .gitignore
├── .env
├── .venv/
├── config/
├── src/
└── main.py
```

Your `.gitignore` could contain:

```
# Python
__pycache__/
*.py[cod]

# Virtual environment
.venv/
venv/
env/

# Environment variables / secrets
.env
.env.*

# VS Code
.vscode/

# OS files
.DS_Store
Thumbs.db
```

Very important for your YAML projects

Be careful with YAML files containing secrets.

For example, do not commit:

```
database:
  username: admin
  password: MySecretPassword
```

Instead use:

```
database:
  username: ${DB_USERNAME}
  password: ${DB_PASSWORD}
```

and store actual secrets in `.env` or a proper secret manager.

9. Check Git status

Run:

```
git status
```

You may see:

```
Untracked files:
  config/
  src/
  main.py
```

This means:

Git knows these files exist, but they haven't been added to the next commit yet.

This is where one of Git's most important concepts comes in.

10. Understand the Git lifecycle

For every change, think about this:

```
Working Directory
  |
  | git add
  ▼
Staging Area
  |
  | git commit
  ▼
Local Repository
  |
  | git push
  ▼
GitHub
```

This is the core Git workflow.

Working Directory

Your actual files:

```
main.py
settings.py
application.yaml
```

Staging Area

Files selected for the next commit.

Commit

A permanent snapshot in your local Git history.

Push

Send commits from your computer to GitHub.

11. Stage your YAML project

You can use the command line:

```
git add .
```

This means:

Add all current changes to the staging area.

Then:

```
git status
```

You should see something like:

```
Changes to be committed:

  new file:   config/application.yaml
  new file:   src/yaml_loader.py
  new file:   src/config_loader.py
  new file:   src/settings.py
  new file:   main.py
```

12. The same thing through VS Code

This is where the VS Code integration becomes useful.

Click:

```
Source Control
```

You will see:

```
Changes
-----
M main.py
U config/application.yaml
U src/settings.py
```

U generally means: **Untracked**

M means: **Modified**

You can click the + beside a file to stage it.

Or use:

```
Stage All Changes
```

VS Code provides these Git operations directly through the Source Control interface. (Visual Studio Code)

13. Create your first commit

After staging the files, enter a commit message.

For example:

```
Initial YAML configuration project
```

Then click:

```
Commit
```

Equivalent command:

```
git commit -m "Initial YAML configuration project"
```

You have now created your first Git version.

14. Understand what just happened

Your project now looks conceptually like:

```
Git Repository  
  
Commit 1  
|  
├─ config/  
├─ src/  
├─ main.py  
└─ .gitignore
```

Later you modify:

```
settings.py
```

You commit again:

```
Commit 2  
|  
└─ Update Settings object
```

Then:

```
Commit 1
↓
Commit 2
↓
Commit 3
↓
Commit 4
```

Git remembers this history.

15. Now create a GitHub repository

Go to GitHub and sign in.

Create a new repository.

For example:

Repository name:

```
yaml-learning-projects
```

I recommend `yaml-learning-projects` rather than something generic like `test` because you're going to keep your learning work there.

Important

Since your local project already contains files and you are going to push it, I recommend creating the GitHub repository without adding an initial README, .gitignore, or license.

You want the GitHub repository to start empty.

You'll push your existing local repository into it.

16. Connect your local Git repository to GitHub

After creating the GitHub repository, GitHub will provide a repository URL similar to:

```
https://github.com/YOUR_USERNAME/yaml-learning-projects.git
```

Back in VS Code terminal:

```
git remote add origin https://github.com/YOUR_USERNAME/yaml-learning-projects.git
```

Now verify:

```
git remote -v
```

You should see:

```
origin https://github.com/YOUR_USERNAME/yaml-learning-projects.git (fetch)
origin https://github.com/YOUR_USERNAME/yaml-learning-projects.git (push)
```

What is origin?

`origin` is simply the conventional name Git gives to your remote repository.

Think:

```
Local Git
|
| origin
▼
GitHub
```

17. Rename your branch to main

Modern repositories commonly use: `main`

Run:

```
git branch -M main
```

Check:

```
git branch
```

You should see:

```
* main
```

The `*` means you are currently on that branch.

18. Push your YAML project to GitHub

Now the exciting part.

Run:

```
git push -u origin main
```

Conceptually:

```
local main branch
|
| push
▼
origin/main
|
▼
GitHub
```

After successful authentication, your files should appear in GitHub.

VS Code can also sync local commits with the remote repository through its Source Control interface. (Visual Studio Code)

19. Authentication with GitHub

On Windows, Git Credential Manager can handle authentication and securely store credentials, so you generally don't need to repeatedly enter credentials. (Microsoft GitHub)

Depending on your setup, when you push for the first time you may get a browser authentication flow.

You may see:

Sign in to GitHub

Complete the authentication.
Then Git can reuse the stored credentials.

20. Verify everything

Go back to GitHub and open:

yaml-learning-projects

You should see something like:

```
yaml-learning-projects
├── config/
│   ├── application.yaml
│   └── development.yaml
├── src/
│   ├── yaml_loader.py
│   ├── config_loader.py
│   ├── settings.py
│   └── __init__.py
├── .gitignore
└── main.py
```

Congratulations.
You have now completed:

```
Windows Git
  ↓
VS Code
  ↓
Local Git Repository
  ↓
GitHub Remote Repository
```

21. Your daily workflow from now on

This is what you'll use almost every day.
Suppose tomorrow you modify:

src/settings.py

and:

config/application.yaml

Your workflow becomes:

Step 1 — Open project

Open your YAML project in VS Code.

Step 2 — Make changes

For example:

```
class Settings:
    ...
```

and:

```
application:
  name: yaml-learning
```

Step 3 — Check changes

```
git status
```

Step 4 — Review changes

```
git diff
```

This shows exactly what changed.

Step 5 — Stage

```
git add .
```

Step 6 — Commit

```
git commit -m "Add structured Settings configuration"
```

Step 7 — Push

```
git push
```

That's it.

22. The same workflow entirely inside VS Code

You don't have to use the terminal every time.

VS Code provides:

```
Source Control
├── Changes
├── Stage
├── Commit
└── Sync / Push
```

So your normal workflow can be:

```
Edit YAML
↓
Save
↓
Source Control
↓
Review changes
↓
Stage
↓
Commit
↓
Sync / Push
```

VS Code's Source Control UI and command-line Git operations work together; you can switch between them without maintaining separate Git histories. (Visual Studio Code)

23. I strongly recommend learning both GUI and commands

Since you're learning toward professional GenAI/AgenticAI development, don't rely exclusively on the VS Code buttons.

Learn both.

Operation	Terminal	VS Code
Check changes	<code>git status</code>	Source Control
See differences	<code>git diff</code>	Click file
Stage	<code>git add .</code>	+
Commit	<code>git commit -m "..."</code>	Commit button
Push	<code>git push</code>	Sync/Push
Pull	<code>git pull</code>	Sync/Pull
Branch	<code>git branch</code>	Branch menu
Clone	<code>git clone</code>	Git: Clone

This will make you much more comfortable when working on servers, Azure DevOps pipelines, CI/CD, Docker environments, etc.

24. VERY IMPORTANT: Pull before starting work

Once your repository is on GitHub, develop the habit:

```
git pull
```

before starting work, especially if you work from multiple machines or collaborate with others.

For example:

```
GitHub
|
| git pull
▼
Your computer
|
| modify
▼
git add
|
▼
git commit
|
▼
git push
|
▼
GitHub
```

VS Code's Sync operation can perform the equivalent remote synchronization, including pulling and pushing. (Visual Studio Code)

25. Branches — learn this next

Don't start doing all your work directly on main once your projects become serious.
For example:

```
main
|
├── yaml-stage-3
├── pydantic-validation
└── config-refactoring
```

For your current YAML learning project, you could eventually have:

```
main
|
├── stage-1-yaml-basics
├── stage-2-yaml-loading
├── stage-3-config
├── stage-3.1-settings
└── stage-3.2-pydantic
```

That becomes especially useful because you can experiment without breaking your stable main branch.

26. Your YAML learning project can become a proper Git project

I'd structure your learning repository approximately like this:

```

yaml-learning/
├── .gitignore
├── README.md
├── config/
│   ├── development.yaml
│   ├── production.yaml
│   └── application.yaml
├── src/
│   ├── __init__.py
│   ├── yaml_loader.py
│   ├── config_loader.py
│   └── settings.py
├── tests/
│   └── ...
├── main.py
└── requirements.txt

```

Then your Git history could look like:

```

main

• Stage 3.2 - Add Pydantic validation
|
• Stage 3.1 - Introduce Settings object
|
• Stage 3 - Application configuration
|
• Stage 2 - YAML configuration loader
|
• Stage 1 - YAML fundamentals
|
• Initial project

```

This is actually a very good way to preserve your learning journey.

27. One important security rule for your YAML projects

Because you're learning configuration management, you'll eventually encounter:

```

database:
  username: xxx
  password: xxx

azure:
  client_id: xxx
  client_secret: xxx

openai:
  api_key: xxx

```

Never push real credentials to GitHub.

Bad:


```
api_key: sk-real-secret-key
```

Good:

```
api_key: ${OPENAI_API_KEY}
```

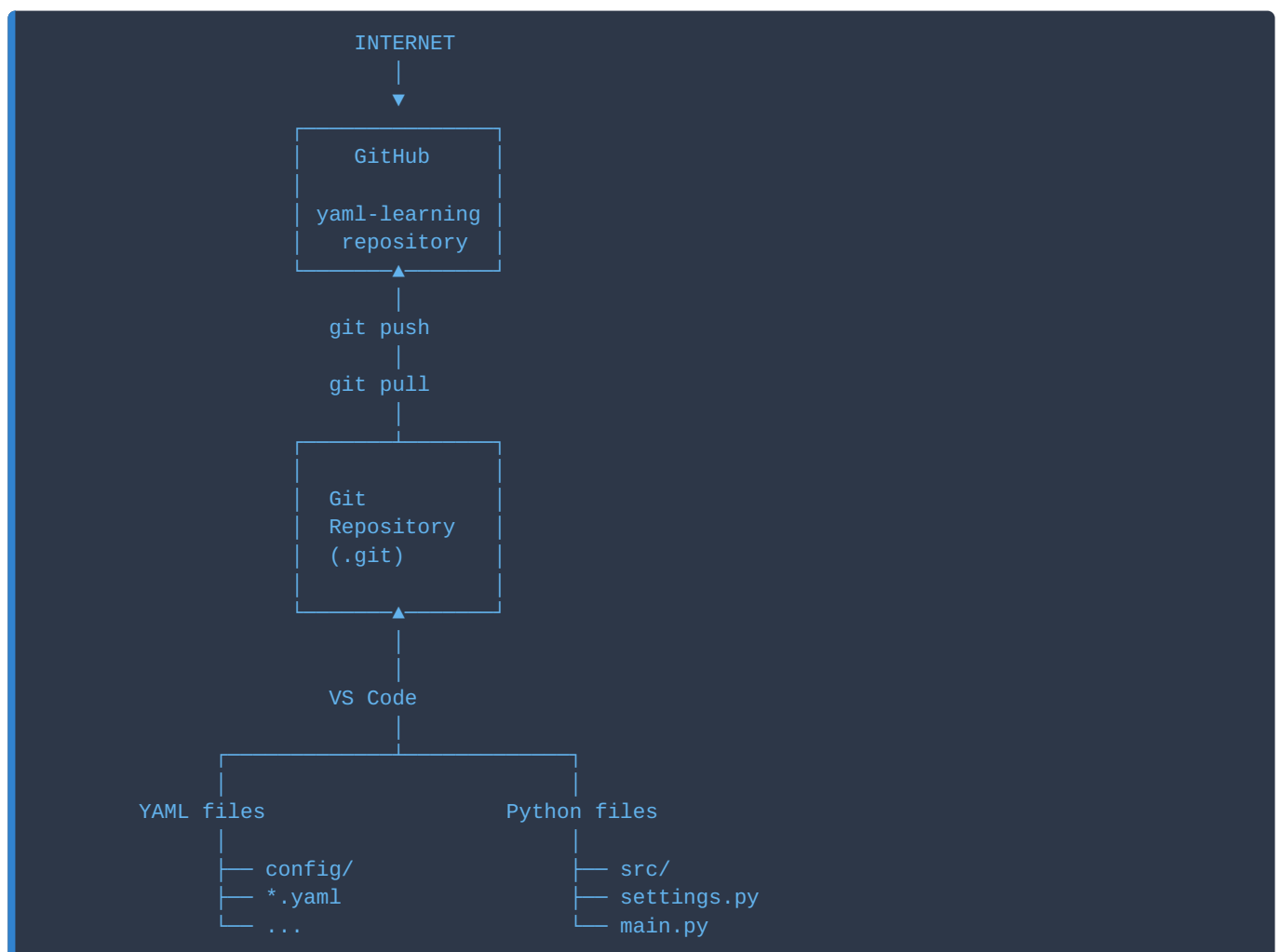
And:

`.env` should normally be included in `.gitignore`.

If a secret is accidentally committed, simply deleting it in a later commit does not necessarily remove it from Git history. Treat exposed credentials as compromised and rotate/revoke them.

28. The complete setup in one picture

Your final architecture should be:



29. Your first practical exercise

I suggest don't just read this. Let's actually set it up.

Do these commands first in your YAML project:

```
git --version
```

then:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

then:

```
git init
```

then:

```
git status
```

Then create your `.gitignore`.

Then:

```
git add .
```

then:

```
git commit -m "Initial YAML learning project"
```

Stop there.

Once that works, we will do the second half:

```
Local Git Repository
  ↓
Create GitHub Repository
  ↓
git remote add origin
  ↓
git branch -M main
  ↓
git push -u origin main
  ↓
Verify project on GitHub
  ↓
VS Code ↔ GitHub daily workflow
```

This staged approach is better for learning than throwing 20 commands at you at once.

The official VS Code documentation also follows essentially this progression: open/initialize a repository → review changes → stage/commit → synchronize with the remote. (Visual Studio Code)

Your Git mental model to remember

```
EDIT
  ↓
git status
  ↓
git add
  ↓
git commit
  ↓
git push
  ↓
GitHub
```

And when starting work:

```
GitHub  
↓  
git pull  
↓  
EDIT  
↓  
git add  
↓  
git commit  
↓  
git push
```

That is the fundamental Git workflow you'll use throughout your YAML, Python, RAG, GenAI and AgenticAI projects.